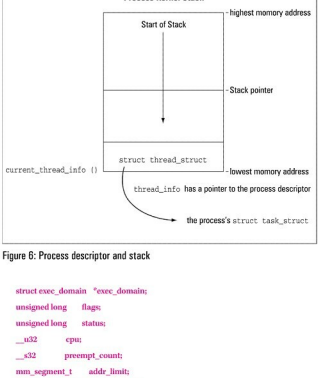

Figure 6: Process descriptor and stack

```
    struct exec_domain  *exec_domain;
    unsigned long       flags;
    unsigned long       status;
    __u32               cpu;
    __s32               preempt_count;
    mm_segment_t        addr_limit;
    struct restart_block restart_block;
    unsigned long       previous_esp;
    __u8                supervisor_stack[0];
};
```

You may note that the task element of the structure is a pointer to a task's **task_struct.**

Runqueues

The fundamental data structure in the scheduler is **runqueue**. For its definition, please see the code (commented) below:

```
struct runqueue {
    spinlock_t       lock;  /* spin lock that protects this runqueue */
    unsigned long    nr_running;     /* number of runnable tasks */
    unsigned long    nr_switches;    /* context switch count */
    unsigned long    expired_timestamp;  /* time of last array swap */
    unsigned long    nr_uninterruptible;  /* uninterruptible tasks */
    unsigned long long  timestamp_last_tick;  /* last scheduler tick */
    struct task_struct  *curr;           /* currently running task */
    struct task_struct  *idle;      /* this processor's idle task */
    struct mm_struct    *prev_mm;      /* mm_struct of last ran task */
    struct prio_array   *active;       /* active priority array */
    struct prio_array   *expired;      /* the expired priority array */
    struct prio_array   arrays[2];     /* the actual priority arrays */
    struct task_struct  *migration_thread; /* migration thread */
    struct list_head    migration_queue;  /* migration queue*/
    atomic_t         nr_iowait; /* number of tasks waiting on I/O */
};
```

If you wish to see the complete code, please look at the **kernel/sched.c** (see struct **runqueue** in the code) file. The **real-timeclasses**' related field in a **runqueue** is given here for your reference:

```
struct rt_rq {
    struct rt_prio_array active;
    unsigned long rt_nr_running;
#if defined CONFIG_SMP || defined CONFIG_RT_GROUP_SCHED
    int highest_prio; /* highest queued rt task prio */
#endif
#ifdef CONFIG_SMP
    unsigned long rt_nr_migratory;
    int overloaded;
#endif
    int rt_throttled;
    u64 rt_time;
    u64 rt_runtime;
    /* Nests inside the rq lock: */
    spinlock_t rt_runtime_lock;

#ifdef CONFIG_RT_GROUP_SCHED
    unsigned long rt_nr_boosted;

    struct rq *rq;
    struct list_head leaf_rt_rq_list;
    struct task_group *tg;
    struct sched_rt_entity *rt_se;
#endif
};

#ifdef CONFIG_SMP
```

Runqueue is basically a list of processes that are scheduled to run in a processor, and there will be one such list for each processor. But a given process will not appear in two lists. You might have noticed that I have referred to a .c file and not a header one. If you call a header file, it can include codes outside of the scheduler. In order to prevent this from happening, we refer to the other file. Two macros are important when it comes to **runqueue**, and they are the macro **cpu_rq** (which returns the pointer to the **runqueue** linked to a particular processor) and the **this_rq()** macro (which points to the current one). There are other macros also that are linked to this (like **task_rq(task)**).

We are done for today. Wait for the next instalment to continue the voyage. Happy kernel hacking! END

By: Aasis Vinayak PG

The author is a hacker and a free software activist who does programming in the open source domain. He is the developer of V-language—a programming language that employs AI and ANN. His research work/publications are available at www.aasisvinayak.com